PRACTICAL - 14

Part A — Practice Programs (Hour 1)

PROBLEM1	1. Assemble and test: build a tiny layered app (a model, a service using an ArrayList, and a menu main), then write two or three JUnit @Test methods checking that adding increases the count and an invalid action throws — practice before testing MiniBank.
CODE	

```
public class Account {
   private String name;
   private double balance;

   public Account(String name, double balance) {
      this.name = name;
      this.balance = balance;
   }

   public String getName() {
      return name;
   }

   public double getBalance() {
      return balance;
   }
}

import java.util.ArrayList;

public class AccountService {
   private ArrayList<Account> accounts = new ArrayList<>();

   public void addAccount(Account account) {
      if (account == null) {
         throw new IllegalArgumentException(
            "Account cannot be null"
         );
      }

      accounts.add(account);
   }

   public int getCount() {
      return accounts.size();
   }

   public Account getAccount(int index) {
```

```
        if (index < 0 || index >= accounts.size()) {
            throw new IllegalArgumentException(
                "Invalid account index"
            );
        }

        return accounts.get(index);
    }
}

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        AccountService service = new AccountService();

        int choice;

        do {
            System.out.println("\n1. Add Account");
            System.out.println("2. View Account Count");
            System.out.println("3. Exit");
            System.out.print("Enter choice: ");

            choice = sc.nextInt();
            sc.nextLine();

            switch (choice) {
                case 1:
                    System.out.print("Enter name: ");
                    String name = sc.nextLine();

                    System.out.print("Enter balance: ");
                    double balance = sc.nextDouble();

                    service.addAccount(
                        new Account(name, balance)
                    );

                    System.out.println("Account added!");
                    break;

                case 2:
                    System.out.println(
                        "Total accounts: " + service.getCount()
                    );
```

```
                break;

            case 3:
                System.out.println("Exiting...");
                break;

            default:
                System.out.println("Invalid choice!");
        }
    } while (choice != 3);

    sc.close();
  }
}


import java.util.Scanner;

public class Main {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    AccountService service = new AccountService();

    int choice;

    do {
      System.out.println("\n1. Add Account");
      System.out.println("2. View Account Count");
      System.out.println("3. Exit");
      System.out.print("Enter choice: ");

      choice = sc.nextInt();
      sc.nextLine();

      switch (choice) {
        case 1:
          System.out.print("Enter name: ");
          String name = sc.nextLine();

          System.out.print("Enter balance: ");
          double balance = sc.nextDouble();

          service.addAccount(
            new Account(name, balance)
          );

          System.out.println("Account added!");
```

```
                break;

            case 2:
                System.out.println(
                    "Total accounts: " + service.getCount()
                );
                break;

            case 3:
                System.out.println("Exiting...");
                break;

            default:
                System.out.println("Invalid choice!");
        }
    } while (choice != 3);

    sc.close();
  }
}


public class AccountServiceTest {

  public static void main(String[] args) {
    addingAccountIncreasesCount();
    addingTwoAccountsIncreasesCountByTwo();
    invalidAccountThrowsException();

    System.out.println("\nAll tests passed!");
  }

  static void addingAccountIncreasesCount() {
    AccountService service = new AccountService();

    service.addAccount(new Account("Om", 1000));

    if (service.getCount() != 1) {
      throw new AssertionError("Count should be 1");
    }

    System.out.println("Test 1 passed");
  }

  static void addingTwoAccountsIncreasesCountByTwo() {
    AccountService service = new AccountService();
```

```
        service.addAccount(new Account("Om", 1000));
        service.addAccount(new Account("Rahul", 2000));

        if (service.getCount() != 2) {
            throw new AssertionError("Count should be 2");
        }

        System.out.println("Test 2 passed");
    }

    static void invalidAccountThrowsException() {
        AccountService service = new AccountService();

        try {
            service.addAccount(null);
            throw new AssertionError("Exception was not thrown");
        } catch (IllegalArgumentException expected) {
            System.out.println("Test 3 passed");
        }
    }
}
```

OUTPUT

```
                                                javac -d out src\model\Account.java src\service\AccountService.java src\app\Main.java test\AccountServi
ceTest.java\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 14>
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 14> java -cp out AccountServiceTest
Test 1 passed
Test 2 passed
Test 3 passed

All tests passed!
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\PRACTICAL 14>
```

PROBLEM2

2. (For Advanced Learners – home) Organise that app into packages, build a runnable JAR, and add a test that saves and reloads its data unchanged.

CODE

```
package model;

import java.io.Serializable;

public class Account implements Serializable {
    private static final long serialVersionUID = 1L;

    private String name;
    private double balance;

    public Account(String name, double balance) {
        this.name = name;
        this.balance = balance;
    }
```

```
    public String getName() {
        return name;
    }

    public double getBalance() {
        return balance;
    }
}


package service;

import model.Account;
import java.util.ArrayList;
import java.io.*;

public class AccountService {
    private ArrayList<Account> accounts = new ArrayList<>();

    public void addAccount(Account account) {
        if (account == null) {
            throw new IllegalArgumentException(
                "Account cannot be null"
            );
        }

        accounts.add(account);
    }

    public int getCount() {
        return accounts.size();
    }

    public Account getAccount(int index) {
        if (index < 0 || index >= accounts.size()) {
            throw new IllegalArgumentException(
                "Invalid account index"
            );
        }

        return accounts.get(index);
    }

    public void saveData(String filename) throws IOException {
        try (ObjectOutputStream out = new ObjectOutputStream(
                new FileOutputStream(filename))) {
            out.writeObject(accounts);
```

```
        }
    }

    @SuppressWarnings("unchecked")
    public void loadData(String filename)
            throws IOException, ClassNotFoundException {
        try (ObjectInputStream in = new ObjectInputStream(
                new FileInputStream(filename))) {
            accounts = (ArrayList<Account>) in.readObject();
        }
    }
}


package app;

import model.Account;
import service.AccountService;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        AccountService service = new AccountService();

        int choice;

        do {
            System.out.println("\n1. Add Account");
            System.out.println("2. View Account Count");
            System.out.println("3. Save Data");
            System.out.println("4. Load Data");
            System.out.println("5. Exit");
            System.out.print("Enter choice: ");

            choice = sc.nextInt();
            sc.nextLine();

            try {
                switch (choice) {
                    case 1:
                        System.out.print("Enter name: ");
                        String name = sc.nextLine();

                        System.out.print("Enter balance: ");
                        double balance = sc.nextDouble();
```

```
                service.addAccount(
                    new Account(name, balance)
                );

                System.out.println("Account added!");
                break;

            case 2:
                System.out.println(
                    "Total accounts: " + service.getCount()
                );
                break;

            case 3:
                service.saveData("accounts.dat");
                System.out.println("Data saved!");
                break;

            case 4:
                service.loadData("accounts.dat");
                System.out.println("Data loaded!");
                break;

            case 5:
                System.out.println("Exiting...");
                break;

            default:
                System.out.println("Invalid choice!");
          }
        } catch (Exception e) {
          System.out.println("Error: " + e.getMessage());
        }

    } while (choice != 5);

    sc.close();
  }
}


import model.Account;
import service.AccountService;

public class AccountServiceTest {
  public static void main(String[] args) throws Exception {
    AccountService original = new AccountService();
```

```
        original.addAccount(new Account("Om", 5000));
        original.addAccount(new Account("Rahul", 3000));

        original.saveData("test-accounts.dat");

        AccountService restored = new AccountService();
        restored.loadData("test-accounts.dat");

        if (original.getCount() != restored.getCount()) {
            throw new AssertionError("Account count does not match");
        }

        for (int i = 0; i < original.getCount(); i++) {
            Account a = original.getAccount(i);
            Account b = restored.getAccount(i);

            if (!a.getName().equals(b.getName())
                    || a.getBalance() != b.getBalance()) {
                throw new AssertionError("Account data does not match");
            }
        }

        System.out.println("Save and reload test passed!");
    }
}
```

manifest.mf

Main-Class: app.Main

OUTPUT

